

Informe de Laboratorio 04

Fundamentos de JavaScript ECMAScript 6+, Manipulación del DOM y Manejo de E

Nota

Estudiante	Escuela	Asignatura
Jeferson Gabriel Huanca Jara jhuancaja@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Introducción al Desarrollo Web Grupo: Código: 2501210

Grupo 4 - Integrantes	Participación
Choquemamani Calisaya José Juan	100 %
Huanca Jara Jeferson Gabriel	100 %
Quispe Rodríguez Ricardo Alcides	100 %
Flores Phuyokawa Christopher Ismael	100 %

Laboratorio	Tema	Duración
04	Fundamentos de JavaScript ECMAScript 6+, Manipulación del DOM y Manejo de Eventos	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	21/09/2026	30/09/2026

1. Tarea

- Aplicar la sintaxis moderna de JavaScript (ES6+): variables `let/const`, funciones flecha, destructuring y métodos de arrays (`map`, `filter`, `reduce`).
- Seleccionar, crear, modificar y eliminar nodos dinámicamente en el Document Object Model (DOM).
- Capturar y procesar eventos de usuario mediante Event Listeners y realizar la validación de datos de entrada en el cliente.
- Implementar la aplicación interactiva de "Gestor de Tareas Universitarias" manteniendo la persistencia de datos mediante la API `localStorage` y serialización JSON.

2. Equipos, materiales y temas utilizados

- Sistema Operativo Ubuntu GNU Linux 24.04 LTS / Windows 11.
- Visual Studio Code 1.95+ con extensión Live Server.
- Navegador Web Google Chrome / Brave con DevTools Console.
- JavaScript ES6+ (ECMAScript 2015+), HTML5, CSS3.
- Git 2.45+ y cuenta institucional en GitHub.
- Tema 08: Fundamentos de JavaScript.
- Tema 09: DOM, Eventos y Validación con JavaScript.

3. URL de Repositorio GitHub

- URL del Repositorio GitHub para clonar o recuperar:
- <https://github.com/Ismael1517/idweb.git>
- URL para el laboratorio 04 en el Repositorio GitHub:
- <https://github.com/Ismael1517/idweb/tree/main/lab%20-04>

4. Actividades con el repositorio GitHub

4.1. Inicialización de la estructura de trabajo para el Laboratorio 04

- Creación del directorio correspondiente al laboratorio 04 e inicialización de archivos fuente.

Listing 1: Creando carpeta y estructura para el Laboratorio 04

```
$cd$HOME/rescobedoq/idweb
$mkdir -p unidad01/lab04$ touch unidad01/lab04/index.html unidad01/lab04/app.js
    unidad01/lab04/style.css
$git add .$ git commit -m "Estructura inicial para el Laboratorio 04: JS y DOM"
$ git push -u origin main
```

4.2. Implementación de la Lógica JavaScript y Manipulación del DOM

- A continuación se presenta la implementación del módulo de gestión de tareas con inserción dinámica en el DOM, manejo de eventos y almacenamiento persistente mediante `localStorage`:

Listing 2: app.js - Módulo de Gestión de Tareas

```
const form = document.querySelector('#todo-form');
const input = document.querySelector('#todo-input');
const list = document.querySelector('#todo-list');

let tasks = JSON.parse(localStorage.getItem('tasks')) || [];

function renderTasks() {
```

```
list.innerHTML = '';
tasks.forEach((task, index) => {
  const li = document.createElement('li');
  li.className = 'list-group-item d-flex justify-content-between align-items-center';
  li.innerHTML = `
    <span>${task.text}</span>
    <button class="btn btn-danger btn-sm"
      onclick="deleteTask(${index})">Eliminar</button>
  `;
  list.appendChild(li);
});
}

form.addEventListener('submit', (e) => {
  e.preventDefault();
  const text = input.value.trim();
  if (!text) return;

  tasks.push({ text, completed: false });
  localStorage.setItem('tasks', JSON.stringify(tasks));
  input.value = '';
  renderTasks();
});

function deleteTask(index) {
  tasks.splice(index, 1);
  localStorage.setItem('tasks', JSON.stringify(tasks));
  renderTasks();
}

document.addEventListener('DOMContentLoaded', renderTasks);
```

Listing 3: Commit: Implementación completa de la lógica de tareas y DOM

```
$git add . $ git commit -m "Agregando logica de manipulacion DOM y persistencia
  localStorage"
$ git push -u origin main
```

4.3. Estructura de directorio del laboratorio 04

```
lab04/
|--- index.html
|--- app.js
|--- style.css
|--- latex/
|   |--- img/
|   |   |--- logo_abet.png
|   |   |--- logo_episunsa.png
|   |   |--- demo_dom.png
|   |--- informe_lab04.pdf
|   |--- informe_lab04.tex
```

5. Commits más importantes

1. **Commit 1:** Maquetación semántica del Panel e integración de estilos institucionales. Se creó la estructura HTML utilizando elementos semánticos de HTML5 e integrando el framework Bootstrap. Se construyó el formulario principal compuesto por campos de entrada para el título de la tarea, selección de materia, selector de fecha límite y nivel de prioridad. Se estableció el nodo contenedor que servira para conectar tareas con java scrip

```

21 .btn-custom {
22   background-color: #7A001E;
23   color: white;
24   font-weight: 600;
25   border: none;
26   border-radius: 8px;
27   padding: 10px;
28   transition: all 0.2s ease;
29 }
30
31 .btn-custom:hover {
32   background-color: #5b0016; /* Guinda más oscuro para efecto hover */
33   color: white;
34 }
35
36 /* Tarjetas y Bordes con estilo Guinda */
37 .card-custom {
38   border: 1px solid #e2e8f0;
39   border-radius: 12px;
40 }
41
42 .btn-custom {
43   background-color: #7A001E;
44   color: white;
45   font-weight: 600;
46   border: none;
47   border-radius: 8px;
48   padding: 10px;
49   transition: all 0.2s ease;
50 }
51
52 .btn-custom:hover {
53   background-color: #5b0016; /* Guinda más oscuro para efecto hover */
54   color: white;
55 }
56
57 /* Tarjetas y Bordes con estilo Guinda */

```

Figura 1: Capturas del Commit 1.

2. **Commit 2:** Maquetación semántica del Panel e integración de estilos institucionales.

Se definió la variable en memoria let tareas para almacenar la colección de tareas en tiempo de ejecución.

Se implementó la recuperación de datos desde el navegador mediante

Se utilizó el operador lógico OR para inicializar un arreglo vacío en caso de que sea la primera ejecución y no existan datos grabados.

```

1 // 1. ESTADO GLOBAL Y PERSISTENCIA
2 let tareas = JSON.parse(localStorage.getItem('tareassua')) || [];
3 let filtroactual = 'todas';
4
5 const formarea = document.getElementById('formarea');
6 const divalertas = document.getElementById('divalertas');
7 const listatareas = document.getElementById('listatareas');
8 const estadisticas = document.getElementById('estadisticas');
9
10 document.addEventListener('DOMContentLoaded', () => {
11   renderizarTareas();
12 });
13
14 function guardarEnLocalStorage() {
15   localStorage.setItem('tareassua', JSON.stringify(tareas));
16 }
17

```

Figura 2: Capturas del Commit 2.

3. **Commit 3:** Captura de eventos, validación de fechas y alertas dinámicas

Prevención de recarga por defecto: Se invocó e.preventDefault() para detener el envío tradicional del formulario HTML y evitar el refresco de la página.

```

57:   divAlerts.innerHTML = `
58:     <div class="alert alert-${tipo} alert-dismissible fade show rounded-3 mb-3" role="alert">
59:       <i class="fa-solid fa-triangle-exclamation me-2"></i> ${mensaje}
60:       <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
61:     </div>
62:   `;
63: }

```

4. **Commit 4:** Construcción dinámica del DOM con Template Literals y Filtros ES6+

Inyección en el árbol DOM: Se insertaron los componentes generados al árbol DOM asignando atributos data-id a los botones de control para su posterior identificación.

```

66 function render(tareas) {
67     listatareas.innerHTML = '';
68
69     const tareasFiltradas = tareas.filter(tarea => {
70         if (!filterActual) === "pendientes") return tarea.completada;
71         if (!filterActual) === "completados") return tarea.completada;
72         return true;
73     });
74
75     if (tareasFiltradas.length === 0) {
76         listatareas.innerHTML = '';
77         <div class="test-center py-5 text-muted">
78             <div class="fa-regular fa-folder-open fa-3x mb-2"/>
79             opko by tareas registradas en esta seccion /p>
80             </div>;
81             actual=estadisticas();
82             return;
83         }
84     }
85
86     tareasFiltradas.map(tarea => {
87         const col = document.createElement('div');
88         col.className = `col-md-6`;
89
90         col.innerHTML = `
91             <div class="card card-outline card-tarea p-3 ${tarea.completada ? 'opacity-75' : ''}>
92                 <div class="d-flex justify-content-between align-items-start mb-2">
93                     <span class="badge ${tarea.completada ? 'bg-secondary' : 'bg-danger'} text-white">
94                         ${tarea.completada ? 'Completada' : 'Pendiente'}
95                     </span>
96                     <small class="text-muted">xi class="fa-regular fa-calendar mr-1"/> ${tarea.fechaCreacion}</small>
97                 </div>
98                 <div class="fw-bold mb-1">${tarea.completada ? 'tarea completada' : ''}>${tarea.titulo}</div>
99                 <p class="small text-muted mb-1">xi class="fa-solid fa-book me-1"/> ${tarea.url}</p>
100                 <div class="d-flex gap-2 pt-2 border-top">
101                     <button class="btn btn-sm btn-outline-secondary"> ${tarea.url}</button>
102                     <button class="btn btn-sm btn-outline-secondary"> ${tarea.url}</button>
103                 </div>
104             </div>
105         `;
106         listatareas.appendChild(col);
107     });
108 }

```

Página 5

```

118 function alternarEstado(id) {
119     guardarEnLocalStorage();
120     renderizarTareas();
121 }
122
123 function eliminarTarea(id) {
124     tareas = tareas.filter(t => t.id !== id);
125     guardarEnLocalStorage();
126     renderizarTareas();
127 }
128
129 function filtrarTareas(tipo) {
130     filtroActual = tipo;
131     document.querySelectorAll('.filter-btn').forEach(btn => btn.classList.remove('active'));
132     event.target.classList.add('active');
133     renderizarTareas();
134 }
135
136 function actualizarEstadisticas() {
137     const total = tareas.length;
138     const completadas = tareas.reduce((acc, t) => t.completada ? acc + 1 : acc, 0);
139     const pendientes = total - completadas;
140
141     estadisticas.innerHTML = `
142     <span class="small fa-bold">
143     <i class="fa-solid fa-list-check me-1"></i>Total: ${total} |
144     <i class="text-danger">Pendientes: ${pendientes}</i></span> |
145     <span class="text-success">Completadas: ${completadas}</span>
146     </span>`;
147 }
148
149
150
151
152
153

```

Figura 4: Métricas acumulativas y manejo de estado.

6. Cuestionario

1. ¿Cuál es la diferencia de comportamiento entre las palabras clave `var`, `let` y `const` respecto al alcance (scope) y al hoisting?

- **Alcance (Scope):** `var` posee un alcance de función (*function scope*) o global, lo que significa que no respeta bloques de control como `if` o `for`. Por el contrario, `let` y `const` tienen alcance de bloque (*block scope*), estando restringidos únicamente al bloque delimitado por llaves `{}` donde fueron declarados.
- **Hoisting (Elevación):** Las tres palabras clave sufren *hoisting*. Sin embargo, las variables declaradas con `var` son inicializadas automáticamente con el valor `undefined`. En cambio, las variables declaradas con `let` y `const` entran en la Zona Muerta Temporal (*Temporal Dead Zone* - TDZ) desde el inicio del bloque hasta que se ejecuta su línea de declaración, lanzando un error de tipo `ReferenceError` si se intenta acceder a ellas previamente.
- **Reasignación y Mutabilidad:** `var` y `let` permiten la reasignación de valores. Por otro lado, `const` exige ser inicializada en su declaración y no permite reasignar la referencia, aunque los objetos o arreglos asignados a `const` sí pueden ser mutados internamente.

2. Explique la técnica de Delegación de Eventos (*Event Delegation*) en el DOM y por qué es más eficiente que asignar listeners individuales a múltiples elementos.

- La delegación de eventos consiste en registrar un único escuchador de eventos (*Event Listener*) en un elemento padre o contenedor en lugar de registrar múltiples listeners en cada uno de sus elementos hijos.
- Esta técnica aprovecha el mecanismo de *Event Bubbling* (fase de propagación hacia arriba), en la cual los eventos disparados en un nodo hijo ascienden jerárquicamente a través de la estructura del DOM hasta sus ancestros. Mediante la propiedad `event.target`, el nodo padre identifica exactamente cuál elemento hijo fue interactuado.
- **Eficiencia:** Optimiza significativamente el uso de memoria y rendimiento del navegador al reducir el número de listeners activos. Además, gestiona automáticamente la interactividad para elementos agregados dinámicamente al DOM en tiempo de ejecución sin necesidad de vincularles eventos de manera individual.

3. ¿Por qué es necesario utilizar `JSON.stringify()` al almacenar objetos complejos en `localStorage` y qué limitaciones tiene este almacenamiento en términos de seguridad?

- **Serialización:** La API de `localStorage` solo admite almacenamiento de datos en formato clave-valor de tipo cadena de texto (*string*). Si se intenta guardar directamente un objeto o arreglo sin serializar, JavaScript lo convertirá a la cadena implícita "[object Object]", perdiendo la información. Por ello, se emplea `JSON.stringify()` para transformar la estructura de datos compleja a una cadena JSON válida, y posteriormente `JSON.parse()` para reconstruirla.
- **Limitaciones de Seguridad y Rendimiento:**
 - a) *Vulnerabilidad a XSS (Cross-Site Scripting):* Cualquier script malicioso inyectado en el cliente puede leer el contenido de `localStorage`, por lo que jamás deben almacenarse tokens sensibles, claves API o contraseñas.
 - b) *Sin cifrado por defecto:* Los datos se guardan en texto plano en el almacenamiento del navegador.
 - c) *Bloqueo síncrono:* Las lecturas y escrituras en `localStorage` son síncronas e impactan el hilo principal de ejecución si se procesan grandes volúmenes de datos (límite promedio de 5 MB por origen).

7. Rúbrica

7.1. Rúbrica del laboratorio 04

- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo con el cumplimiento de los criterios definidos.

Tabla 1: Niveles de desempeño

Puntos	Nivel			
	Deficiente (0-10)	En Proceso (11-14)	Bueno (15-18)	Excelente (19-20)
7.0 (35 %)	0 - 3.5	3.85 - 4.9	5.25 - 6.3	6.65 - 7.0
6.0 (30 %)	0 - 3.0	3.3 - 4.2	4.5 - 5.4	5.7 - 6.0
4.0 (20 %)	0 - 2.0	2.2 - 2.8	3.0 - 3.6	3.8 - 4.0
3.0 (15 %)	0 - 1.5	1.65 - 2.1	2.25 - 2.7	2.85 - 3.0

Tabla 2: Rúbrica de evaluación analítica para el Laboratorio 04

Criterio y Descripción		Puntos	Chec- klist	Estudian- te	Profesor
1. Manipulación del DOM y Eventos (35 %)	Crea, modifica y elimina nodos del DOM eficientemente. Maneja eventos usando <code>e.preventDefault()</code> y funciones de actualización impecables.	7.0	X	6.0	
2. Lógica JS y Métodos ES6+ (30 %)	Estructura el código utilizando <code>let/const</code> , arrow functions, destructuring y métodos de arreglos (<code>map</code> , <code>filter</code>) con alta legibilidad.	6.0	X	5.0	
3. Persistencia LocalStorage (20 %)	Persiste y recupera el estado completo de la app mediante <code>localStorage</code> y serialización JSON sin fallas de inicialización.	4.0	X	4.0	
4. Cuestionario Teórico (15 %)	Respuestas rigurosas exponiendo conceptos de scope, hoisting, delegación de eventos y JSON con vocabulario técnico perfecto.	3.0	X	3.0	
Total		20.0		18.0	

8. Referencias

- Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
- MDN Web Docs. (2025). *Document Object Model (DOM) & Events*. Developer Mozilla. https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model
- Simpson, K. (2015). *You Don't Know JS: ES6 & Beyond*. O'Reilly Media.